Java Fundamentals

Lab 1

What is Java

- Java is a **high-level, class-based, object-oriented programming language**.
- Java is its **Write Once, Run Anywhere (WORA)** language.

JDK vs. JRE vs. JVM

- **JDK (Java Development Kit):** The JRE + Compilers/Debuggers. Everything you need to create, modify, compile, debug, test, and run
- **JRE (Java Runtime Environment):** The JVM + libraries.
  - To run Java without modification or creation
  - Java standard library(string, math, exception, io…)
- **JVM (Java Virtual Machine):** The engine that actually runs the code.
  - To execute Java code
  - Loads bytecode from class files to memory and runs your program.
  - Write once and run everywhere

Why Java

- Used to develop:

  Web applications: Spring boot, Hibernate, Java server face

  Desktop Applications:JavaFx, Swing, SWT

  Android (Mobile application development):Android SDk, Kotlin and flutter

  Gaming:jMonkeyEngine

  Enterprise Application: Spring framework, Hibernate, java EE

How java works?

Java is unique because it uses a **two-step** process: it is both compiled and lnterpreted.**Step 1: Compilation**

- The programmer writes code in a .java file.
- The **JDK (Java Development Kit)** uses a compiler called javac.
- javac converts the code into **Bytecode** (a .class file).

...

Step 2: Execution

- The Bytecode is platform-independent (it can run anywhere).
- The **JVM (Java Virtual Machine)** reads the .class file.
- The **JIT (Just-In-Time) Compiler** inside the JVM converts Bytecode into machine code that your specific CPU understands.

Basic Syntax

```
public class Main {   // Class declaration

 // Main method - Program starts here

    public static void main(String[] args) {   # to  accept  inputs from the command
line.

        System.out.println("Hello, Java Lab!");  #System (the computer), out (the output
stream), println (print a line of text).

   }

}
```

Exercise

1. Open your IDE.
2. Create a new project.
3. Create a class named Greeter.
4. Modify the code to print: *"Hello, [Your Name]! Welcome to Java Lab."*

Variables, Data Types, and Operators

Variables: named placeholders in computer memory.

Syntax: type name value;

Type: what kind of data it holds

Name: how do we find the placeholder

Value: the actual content

Data Types

instructions to the computer about how to interpret a specific piece of data and how much memory to reserve for it.

Primitive datatypes vs non primitive(reference dt)

Int: integer String: for text

double : decimal Arrays: Collections of data

Char: single character **Classes/Objects:** Custom types

cont…

Examples:

```
int studentsCount = 30;

double gpa = 3.8;

char section = 'B';

boolean isLabDay = true;

String labName = "OOP Fundamentals";
```

Exercises

1. Write a program that asks the user name and age and display the result.
2. Declare variables of diferent datatypes and display thies va;lues

Exercise

The Student Profile Generator

Create a new Java class named StudentProfile. Your program should store student information , calculate your age by subtracting from current year and display a formatted report.

NB; student information should be provided from keyboard

con…

Store Receipt Generator

A student is buying 3 items from the campus store. The program needs to calculate the subtotal, apply a 5% student discount, and display the final total.

Your program must display a receipt that includes:

- The names and prices of all three items.
- The calculated Subtotal.
- The Discount amount (specifically how much was taken off).
- The Final Total.

Conditional Statements

allow your program to make decisions based on certain criteria.

The if-else Structure

This is used when you have two or more paths to choose from.

```
if (condition) {
    // Code to run if condition is true
} else {
    // Code to run if condition is false }
```

The `switch` Statement

The `switch` statement is a cleaner way to write a multi-way branch when you are comparing a **single variable** against several **constant values**.

```
switch (variable) {
    case value1:
        // code
        break;
        default:
        // code if no cases match }
```

Examples

The Quadrant Finder (Beginner)

Write a program that takes an integer angle (0 to 360) and tells the user which quadrant it falls in:

Rules:

- 0-90: Quadrant I
- 91-180: Quadrant II
- 181-270: Quadrant III
- 271-360: Quadrant IV

The Grade Converter

Take a numerical score (0-100) and convert it to a letter grade:

Rules:

- 90+: A
- 80-89: B
- 70-79: C
- Below 70: F

Constraint: If the user enters a number greater than 100 or less than 0, print "Invalid Score."

Loops

The for Loop

The for loop is best used when you **know exactly how many times** you want to run a block of code.

```
for (initialization; condition; update) {
    // Body of the loop
}
```

The while Loop

The while loop is used when the number of iterations is **not known in advance**. It repeats as long as a boolean condition remains true.

```
while (condition) {
    // Body of the loop
    // (Ensure the condition eventually becomes false!)
}
```

The do-while Loop

The do-while loop is a variation of the while loop. The key difference is that it **always executes the code block at least once** before checking the condition.

```
do {
    // Body of the loop
} while (condition);
```

Examples

Q. The Multiplication Table

Write a program that asks the user for a number (e.g., 7) and prints the multiplication table for that number from 1 to 10.

Exercises

Q. The Security PIN Lockout

Write a program that simulates a secure login.

1. Create a constant integer variable named SECRET_PIN (e.g., 1234).
2. Give the user a maximum of **3 attempts** to enter the correct PIN.
3. **Rules:**
    - If the PIN is correct, print "Access Granted" and exit the loop immediately.
    - If the PIN is incorrect, tell the user how many attempts they have left.
    - If they fail 3 times, print "Security Alert: Account Locked."

Q. The ride share price calculator:

Construct a fare calculator that takes two inputs: **Distance (km)** and **Hour of Day (0–23)**.

1. **Base Price:** Every ride starts at **120Birr.**
2. **Distance Fee:** Add **7birr** for every kilometer traveled.
3. **Rules**
    - If the hour is between 7-9 or 16-19 at night, add a flat **50birr** additional charge..
    - **Night Shift:** If the hour is between23 and 5 the **entire final price** should be increased by **50%** (multiplied by 1.5).

Validate your distance and time.

…

Write a java code to generate a horizontal grid of multiplication facts from 1 to 10.

Exercises

Q: Guess My Number" Game

1. Use the Random class to generate random numbers
2. Use a while loop to keep asking the user for a guess.
3. If the guess is too high, print "Too high!"
4. If the guess is too low, print "Too low!"
5. If they guess correctly, print "You got it!" and exit the loop.

Arrays

The Temperature Tracker :

- Create an array of double values and initialize it with 7 temperatures.
- Write a loop to calculate the **average** temperature for the week.
- Find and print the highest temperature in the array without using a built-in sort method.

ArrayList

1. Create an ArrayList of Strings.
2. Use a while loop to let the user type in items one by one.
3. If the user types "done", stop the loop.
4. If the user types "remove", remove the last item added to the list.
5. Print the final list and the total number of items.